Staged Information Exposure: Capability Gateways for Multi-Agent Systems

Maximilian Armuss
Student @ TUM School of CIT
ge37yow@mytum.de

Max Huppertz
Student @ TUM School of CIT
max.huppertz@tum.de

Dong Sicheng
Student @ TUM School of CIT
sicheng.dong@tum.de

Li Penglin
Student @ TUM School of CIT
ge74mix@tum.de

Dr. Maximilian Wich
Supervisor @ TUM School of CIT
maximilian.wich@zeiss.com

Dr. Alecio Binotto
Supervisor @ TUM School of CIT
alecio.binotto@zeiss.com

***Abstract*—Today's multi-agent systems grow by adding more tools and peer agents. However, current frameworks often expose semantic capability descriptions, protocol formats, and execution details at the same time. This forces agents to plan and handle protocol mechanics in a single context. We formalize this as a problem of poorly staged information exposure.**

To address it, we propose a unified capability gateway that mediates both tools and peer agents through a common capability layer. The gateway lets agents plan on semantic capability information first and delays execution-specific details until they are needed. It also provides a central point for configuration and governance across protocols. In a vulnerability-detection benchmark, gateway-based architectures improve task success, token efficiency, and latency over simple protocol-exposure baselines. We further show that retrieval-based capability selection becomes especially useful as the environment grows, because it prevents irrelevant capability information from entering the context.

I. Introduction

Recent years have seen rapid progress in large language model agents and increasingly complex multi-agent ecosystems. Protocols such as the Model Context Protocol (MCP) and Agent-to-Agent (A2A) promise interoperable tool use and agent communication. In practice, however, current frameworks still tend to expose too much information too early. Semantic capability descriptions, protocol formats, and execution details are often injected into the agent context upon initialization.

We argue that this is an architectural problem. Classical agent literature characterizes agents as autonomous entities that act based on their own decision-making process[27]. Recent work such as MCP-Zero[7] similarly argues that exposing full tool schemas to the model creates context overhead and reduces decision autonomy. In heterogeneous ecosystems, this problem becomes even more pronounced. Agents must reason not only about which capability they need, but also about protocol-specific execution patterns for both tools and peer agents. This increases context bloat, raises the risk of protocol confusion, and can degrade end-to-end performance.

To address this, we propose a unified capability gateway that mediates both MCP tools and A2A peer agents through a common capability layer. Its core purpose is staged information exposure, so that agents can plan on semantic capability information before execution-specific protocol details enter the context. At the same time, the gateway also acts as a central configuration and governance point, where routing, protocol integration, and cross-cutting control mechanisms can be handled consistently.

We evaluate this idea in a realistic vulnerability-detection multi-agent system. We compare two direct framework-based baselines against two gateway-based architectures and analyze task success, latency, token consumption, and common failure patterns. We further study retrieval-based capability selection under growing capability load. Our results show that gateway-based architectures are more robust and efficient in the evaluated setting, while retrieval becomes especially useful as the surrounding environment grows.

Our main contributions are:

- We propose a unified capability gateway for heterogeneous multi-agent systems that enables staged information exposure across planning and execution.
- We show empirically that gateway-based architectures outperform direct protocol exposure baselines in robustness, token efficiency, and end-to-end task success in our benchmark setting.
- We show that retrieval-based capability selection becomes most useful as the environment grows, as it enables agents to fetch only relevant information.

II. Related Work

Agent decision-making and task planning lie at the core of both single-agent and multi-agent success. Foundational cognitive architectures such as CoALA [29] and ReAct [31] underpin most modern agent implementations but assumed relatively small external tool sets. As agents face real-world tasks, this assumption breaks down: long contexts cause information loss [15] and task distraction [28], incorrect tool Retrieval accounts for over half of task failures [20], and even state-of-the-art models suffer significant performance limitations without dedicated retrieval optimizations [19]. This has driven

a growing body of work on effective tool selection from large and evolving resource sets [24, 26, 10, 30].

Much of this literature targets MCP, which established itself early as a widely adopted tool-integration standard [6]. Approaches to improving MCP tool discovery can thereby be divided into LLM-internal and external retrieval solutions. Internal approaches such as ToolGen [30] represent each tool as a token and fine-tune the model to retrieve them. While effective even at scale, this requires fixed token-level representations and explicit fine-tuning constraints ill-suited to rapidly evolving tool ecosystems.

External approaches decouple retrieval from the model and can be categorized by strategy. Proxy-based solutions such as MCP Bridge or ToolRegistry [1, 5] implement a lightweight gateway mediating communication on behalf of the agent. Semantic retrieval approaches [10, 8, 7, 18, 17, 9] introduce vector-based selection of relevant tools at query time and differ primarily in their embedding strategies, retrieval metrics, or specialized description and execution formats. Lastly, graph-based approaches [22, 21] encode agent and tool relationships in knowledge graphs to discover previously unseen connections, but require explicit graph construction and assume agents can freely share their tools, which rarely holds in industry settings with independent peers.

Collectively, this literature reflects a shift toward agents that dynamically fetch resources based on semantic relevance rather than passively receiving static capability dumps. This resonates with our core argument that agents should govern their own capability acquisition, progressively gathering only the information relevant to their current task instead of being overloaded upfront.

Yet most work remains focused on tool calling alone. Real-world settings, however, also require inter-agent communication. The proliferation of agent-communication protocols [6], most notably A2A, compounds the existing challenges: Context bloat, protocol confusion, and retrieval errors all worsen when multiple protocols must coexist. Our work addresses this gap by proposing a gateway that abstracts both tools and peer agents into a unified capability model, hiding protocol-specific details behind a common interface. This design not only scales more gracefully across protocols than direct exposure but also integrates semantic retrieval at the gateway level, combining insights from proxy-based mediation and tool-retrieval research into a single architecture.

III. Background: MCP & A2A

Modern agents no longer interact with a small, fixed set of tools, rather they must also communicate with peer agents to complete complex tasks successfully. The resulting multi-agent systems enable the decomposition of complex problems among specialized agents, each contributing distinct capabilities and domain knowledge within a collaborative network [14]. Two protocols currently dominate this landscape: MCP for tool invocation and A2A for agent-to-agent communication.

A. Model Context Protocol

MCP is an open protocol that standardizes the interface through which large language model applications discover and invoke external capabilities [2]. It adopts a client-server architecture in which an MCP host connects through lightweight clients to one or more MCP servers, each exposing a set of *tools*, *resources*, and *prompts*. A tool definition comprises a unique name, a natural-language description intended to inform the model's selection process, and a JSON schema specifying the expected input parameters. At runtime, the full set of available tool definitions is typically injected into the LLM's context window, providing the information necessary for the model to determine which tool to invoke and how to construct a valid call. MCP thereby decouples capability access from backend-specific implementation details, yet delegates both discovery and execution reasoning to the consuming model within a single context.

B. Agent-2-Agent Protocol

A2A, on the other hand, addresses the discovery of and collaboration between autonomous agents [11]. Its central abstraction is the *Agent Card*, a machine-readable metadata document that each agent publishes to advertise its capabilities. An Agent Card contains structured fields describing the agent's identity, natural-language descriptions of its competencies, supported interaction modalities, and authentication requirements. In contrast to MCP tool definitions, which specify fine-grained callable functions and their parameter schemas, Agent Cards characterize coarse-grained, goal-level capabilities of an entire agent without prescribing its internal implementation. Consuming agents use these cards to identify suitable peers, negotiate task delegation through structured message exchanges, and coordinate multi-step workflows via shared artifacts. Importantly, A2A is explicitly agnostic to the internal architecture of participating agents: An A2A-registered agent may rely on MCP for its own tool access while exposing only high-level capabilities through its Agent Card. The two protocols thus occupy distinct but complementary roles, where MCP governs the *tool plane*, defining how individual functions are described and invoked, whereas A2A governs the *collaboration plane*, defining how agents advertise, discover, and delegate work to one another [6].

IV. System Architectures

We implemented four architecture variations (*Simple*, *Simple-Crew*, *Gateway*, and *Gateway-RAG*). The first two follow conventional multi-agent coordination patterns, while the latter two represent a deliberate architectural departure by introducing our unified capability abstraction and gateway design. All four solve the same problem, which we define in the next section. However for now, we treat each architecture as task-agnostic and focus on interaction patterns.

A. Simple Architectures

1) Simple Architecture: The simple architecture serves as our baseline and does not employ any optimizations for tool

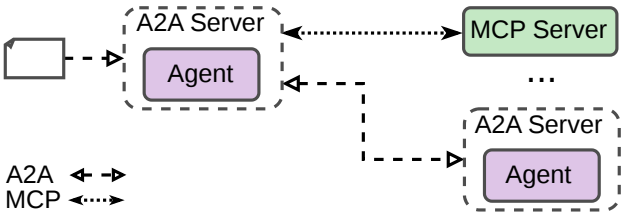


Fig. 1. Simple Architecture: Agents handle both MCP and A2A

or agent retrieval. It can be viewed as the standard implementation path suggested by existing frameworks and protocols.

Each agent is exposed via the A2A protocol so that it can be invoked by other agents. A single agent receives an explicit configuration that determines its environment, namely which MCP servers it can interact with and which peer agents it can communicate with via A2A. All previously described information about tools and agents is loaded into the agent's context upon first connection to the respective MCP or A2A server. This means that all semantic and technical details for all tools and agents accumulate in the agent's context from the outset, and the agent bears the full responsibility of producing valid outputs that either its MCP or A2A client can parse.

This design leads to two compounding problems. First, the agent's context is populated with the full set of available tool descriptions and agent specifications, regardless of their relevance to the immediate planning task. This phenomenon is commonly referred to as *context bloat*. Second, the agent must manage two distinct protocol conventions simultaneously, which increases the likelihood of *protocol confusion*, where outputs become syntactically invalid for the intended client.

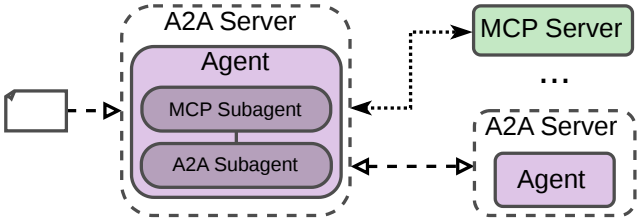


Fig. 2. Simple-Crew Architecture: Agents are divided into crews of protocol-specific subagents

2) Simple-Crew Architecture: A natural first improvement is to split the responsibilities within an agent so that a given A2A-exposed agent is internally composed of two sub-agents: One dedicated to MCP tool retrieval and invocation, and another dedicated to communicating with peer agents via A2A.

This separation allows each sub-agent to specialize on a single protocol. Reducing the protocol confusion creates a new coordination overhead: The sub-agents must collaborate as a team, and each must understand at what stage it should take the lead, despite only having partial visibility into the full set of the agent's capabilities. While this problem is manageable when sub-agent tasks follow a known, relatively strict ordering, this implies that the architecture requires either careful manual specialization of the agent configuration or sufficiently capable sub-agents that can autonomously determine when to hand off control.

Crucially, the problem of context bloat remains unsolved in this architecture. Both sub-agents still receive the full tool descriptions or agent cards for their respective protocols. Instead the bloat is merely distributed across sub-agents that now additionally require coordination context about their partner.

B. Simple Architecture Discussion

Both simple architectures suffer from context bloat, and differ primarily in how they handle protocol confusion: The *Simple* architecture exposes it fully to a single agent, while the *Simple-Crew* variant trades it for coordination overhead. Moreover, both designs scale poorly as additional protocols emerge, since each new standard introduces further conventions and error semantics that agents must manage directly. These limitations motivate a fundamental shift away from direct protocol exposure toward a mediated interaction pattern, which we present next.

C. Gateway Architectures

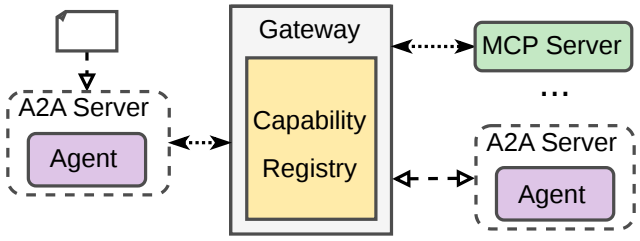


Fig. 3. Gateway Architecture: Agents communicate through protocol-handling gateway with capability abstraction

1) Gateway Architecture: The gateway architecture constitutes a central contribution of this paper and can be portrait as a refined proxy approach as introduced by the MCPBridge [1] or ToolRegistry [5]. In this variation, agents are no longer directly connected to MCP servers or peer agents via A2A. Instead, the gateway mediates all communication. It receives tool descriptions and agent cards from the underlying ecosystem and maps them into a unified capability data model, which constitutes a central abstraction of our system.

Staged Information Exposure: Both MCP tool descriptions and A2A Agent Cards intermingle natural-language descriptions of what a tool or agent can do with protocol-specific formats, authentication requirements, and other technical details. We argue that conflating these two categories of information is architecturally harmful. Agents benefit from reasoning about capability semantics during planning and selection, while protocol-level execution details should only enter the context once a capability has been selected and invocation is imminent. This principle, which we refer to

as *staged information exposure*, motivates a strict phasing of the information an agent receives. Rather than exposing all available metadata upfront, the gateway produces two representations for each capability: The *capability summary*, which contains only semantic information about what a capability does, and the *capability detail*, which additionally includes all technical specifics required for execution. As prior work has indicated, correct capability retrieval is essential for task success [20], and keeping the planning context free of irrelevant technical detail is designed to support exactly this.

The gateway exposes a deliberately thin API consisting of three endpoints that allow agents to:

- Retrieve all capability summaries for the tools and agents available to them
- Retrieve the full details of a specific capability given its unique identifier
- Execute a specific capability with either a structured parameter dictionary or a plain-text handoff

Compared to MCPBridge [1] and ToolRegistry [5], our gateway can handle tool invocation and agent communication protocols. Additionally our API is thiner and we support access both via a standard REST interface and via three lightweight MCP meta-tools. We intentionally designed the API to be as thin yet complete as possible, placing agents back into the regime that their underlying cognitive architectures were designed for. Most notably, ReAct [31] and similar frameworks assume reasoning over a very small tool set, and our design restores exactly this condition.

This design fundamentally changes how agents interact with their environment. Instead of receiving all available tools and Agent Cards alongside the task input and system prompt at the start, the agent's context is initially kept deliberately small. The agent can therefore focus on understanding the concrete task requirements and its behavioral constraints first. Only after reasoning about this foundational information does the agent retrieve capability summaries and decide, based solely on semantic descriptions, which capabilities are best suited for each subtask and in which order to invoke them. As we will show later, this initial freedom and the semantic decision-making it enables, independent of protocol or technical details, helps achieving a substantially higher task success.

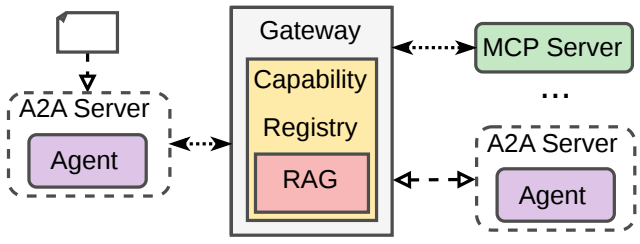


Fig. 4. Gateway-RAG Architecture: Gateway is extended with semantic capability retrieval

2) Gateway-RAG Architecture: Given the already competitive performance of the gateway architecture and inspired by the growing body of literature on semantic tool retrieval [10, 7, 8, 17, 18, 9], we implement a second gateway variation with a modified discovery mechanism. Instead of allowing agents to retrieve all capability summaries at once, the updated gateway API provides an endpoint that returns the top-k capability summaries for a given search query formulated by the agent. The query is embedded and compared against all pre-embedded capability descriptions stored in the gateway's vector database.

This approach grants agents even greater authority over their own context by letting them semantically articulate the capabilities they need and receiving only those that best match. Additionally the top-k parameter can be increased by the agent itself if the initial results with conservative default value are insufficient, preserving even greater agent autonomy over the retrieval process. The benefit becomes especially pronounced in imperfectly configured environments, that is, settings where more tools or peer agents are available than are required for the task at hand. This is typical when agents are deployed as generalists or interact with large external tool servers. In such cases, well-articulated queries combined with a low top-k allow the agent to avoid ever accumulating task-irrelevant context, a problem that persists in all other architecture variations that do not make use of semantic retrieval. The resulting shorter and more focused contexts naturally mitigate two well-documented failure modes of long-context language models: Relevant information being lost within a large context window, and model attention being diverted by irrelevant material [15, 28].

D. Gateway as a central instance

Beyond planning quality, the gateway architecture also yields structural advantages for system operation. Because all protocol-specific logic is consolidated behind a single mediation layer, the gateway serves as a single point of configuration. Adding support for a new protocol requires changes only within the gateway, while agents continue to interact through the same thin API with the capability abstraction. Where direct protocol exposure causes failure modes to grow superlinearly with each additional standard, the gateway absorbs this complexity and presents a stable interface regardless of how many protocols operate underneath.

Most importantly, the gateway does not trade capability for convenience, it combines the best of both worlds: Protocol advantages are preserved, but protocol handling is no longer the caller's responsibility.

Furthermore, the gateway also provides a natural enforcement point for cross-cutting concerns such as access control, rate limiting, audit logging, and input validation. Without centralization, these governance functions would have to be replicated across every agent or embedded into individual protocol clients, leading to fragile and inconsistent enforcement. The gateway allows operators to define and update such policies in one place, with consistent application across all capability invocations.

In other words, the gateway is not merely a proxy. It restructures information exposure across decision phases and also acts as an interoperability and governance boundary.

V. Evaluation Methodology

A. Task Environment

We evaluate our proposed architectures on a vulnerability matching and notification pipeline. The system comprises three agents, each with access to dedicated MCP tool servers, that must collaborate to process a given vulnerability identifier:

1) The *osv lookup agent* retrieves detailed vulnerability information, including affected packages and version ranges, from the official open source vulnerability (*OSV*) database [23].
2) The *matching agent* compares this information against the organization's software bill of materials (SBOM) to identify affected systems and resolves the responsible owner from a separate registry.
3) The *notification agent* compiles the gathered data into a structured JSON payload containing the affected system, matched vulnerability, specific package and version, and the contact person to be informed.

No single agent possesses all information required to complete the task, making the pipeline a suitable testbed for evaluating multi-agent coordination. We deliberately chose this domain because established vulnerability scanners can process the same inputs deterministically, providing a reliable ground truth against which we can verify whether agents collaborate effectively. The contribution of this work is not the underlying system itself but the architectural patterns it evaluates. Our proposed architectures are task-agnostic, and the vulnerability matching scenario serves as an exemplary environment that offers both the data richness and the verifiable outcomes needed for controlled evaluation.

B. Data and Scenario Generation

We draw SBOM inputs from *A Dataset of Software Bill of Materials for Evaluating SBOM Consumption Tools* [13], which contains 46 Maven SBOM files. We decompose these SBOMs into individual package entries to form a shared package pool of approximately 800 packages. For each package, we query known vulnerabilities to determine which CVEs affect it and at which version ranges. This decomposition enables flexible scenario generation, where we reassemble realistic package distributions from the shared pool while controlling for specific risk profiles. Our implementation exposes a configuration interface that determines the number of scenarios, their size, and their risk distribution. We instantiate 20 scenarios with diverse profiles, each containing approximately 20 packages (average SBOM length in dataset), and average metrics over these to mitigate potential bias. Additionally, synthetically generated system-owner data using ChatGPT-5.2, as it is not a semantically crucial component but rather relevant for a field-based comparison against the ground truth.

C. Implementation

To implement the pipelines (*Simple* and *Simple-Crew*), we use the CrewAI framework [4]. This choice is motivated by two practical considerations:

1) CrewAI provides native A2A SDK integration, required for evaluating systems that rely on standardized inter-agent communication [3]
2) Its notion of multi-agent *crews* aligns naturally with our *Simple-Crew* pipeline design

To ensure reproducibility, we pin CrewAI to version $1.6.1$ in our environment configuration. This version was the latest available PyPI release at the start of our implementation, thus any framework-specific errors we report may be resolved in newer releases [25].

The *Gateway* and *Gateway-RAG* pipelines are implemented from scratch using PydanticAI agents and an in-house gateway layer integrating the standard MCP and A2A SDKs behind a unified capability abstraction.

D. Evaluation Suite

Given a list of scenarios according to configuration, the bench

1) Generates SBOMs with the demanded risk profiles
2) Creates ground truth notification data in the same format as the expected notification tool output
3) Executes the full pipeline scenario by scenario
4) Collects and computes end-to-end metrics for each run
5) Averages over all scenario-specific metrics

This end-to-end design keeps the evaluation pipeline-agnostic, and thus ensures fair measurement across all systems.

E. Metrics

We report one effectiveness metric and three efficiency metrics to capture both end-to-end task success and the cost of achieving it. Because our systems are multi-stage and tool-augmented, we focus on metrics that remain interpretable under orchestration failures, partial handoffs, and iterative tool-use.

a) Notification scoring: Our primary metric is `Task Score`, which measures how well the final notification payloads match the ground truth on a field-by-field basis. Each expected notification is evaluated on 8 fields with equal weight. We compute a normalized score in $[0, 1]$ by summing correct fields and dividing by the number of expected fields across all expected notifications:

$$\text{Task Score} = \frac{\sum \text{correct fields}}{\sum \text{expected fields}}$$

This definition makes the metric sensitive to end-to-end failures: If a run times out or an upstream agent drops a critical detail during a handoff, fewer fields match and the score decreases accordingly. Missing expected notifications contribute zero correct fields while still increasing the denominator, resulting in full penalty. We intentionally do not penalize spurious extra notifications, as false alarms are less critical

in a vulnerability context. Additionally, any punishment ratio relative to missing notifications would be arbitrary, as there is no principled basis for choosing one.

In a multi-agent pipeline where no single agent holds all required information, this metric effectively reflects the quality of collaboration: The task score can only be high if each agent contributes its part correctly and passes the right information downstream.

b) Token consumption: We report `Tokens_in` as the total number of input tokens consumed end-to-end, aggregated across all model context windows over the full run. Since input token counts are heavily influenced by the amount and size of provided data (e.g. SBOM size and tool descriptions), we primarily interpret relative trends between architectures rather than absolute numbers. `Tokens_out` measures the total generated output tokens and serves as a complementary signal, helping to distinguish context bloat caused by larger inputs from bloat caused by model verbosity. Notably, even small differences in token consumption early in the pipeline have recursive impact, as each output token is reused in the next reasoning iteration and may be carried forward to subsequent agents. Beyond their diagnostic value, both metrics also have direct practical relevance, as token consumption is the primary cost driver in production deployments of language model based systems.

c) Latency: `Latency` measures end-to-end runtime, capturing both model inference time and framework-specific orchestration overhead. Similar to the token metrics, latency is particularly sensitive to iterative tool-call loops: Repeated attempts increase both runtime and context usage, which is reflected in `Latency` and `Tokens_in`. We therefore use latency to quantify the practical cost of architectural choices beyond raw generation speed.

F. Models and Pipeline Alignment

All runs use the OpenAI model variants `gpt-5`, `gpt-5-mini`, and `gpt-5-nano` from the same 2025-08-07 snapshot via a fixed Azure endpoint. In the case of *Gateway-RAG*, embeddings are created with OpenAI's `text-embedding-3-large`. Evaluation conditions are held constant across architectures: Task inputs, ground truth, agent roles, and authentication are identical. Prompts are kept as similar as possible, though small adaptations are necessary due to architectural differences. Both pipeline types implement an iterative tool-use pattern in which observations are reflected on before producing a final output. In the CrewAI baselines, this pattern is reinforced by framework-managed prompt templates, with tool interfaces exposed directly at agent initialization [12]. The gateway architectures instead expose an interface to fetch capability information on demand. To avoid round-trip overhead from retrieving capabilities one by one, we instruct gateway agents to perform *recipe planning*: The agent decomposes its task into sub-steps, identifies which capabilities are likely needed, and batches discovery and execution requests accordingly. This is a best-effort alignment of two fundamentally different interaction patterns, preserving overall comparability at the task and environment level.

G. Environment Scaling Experiment

The preceding experiments assume a perfectly configured environment where only task-relevant capabilities are available. In practice, however, connecting an MCP server typically exposes its full tool suite rather than a curated subset, meaning agents must operate in environments that contain domain-relevant but task-irrelevant capabilities.

To study this effect, we introduce a controllable MCP server that incrementally adds such capabilities to the environment. Each additional tool carries a plausible vulnerability-management description but is not required to solve the actual task. We scale the number of these additional tools up to eight per agent and compare *Gateway* vs. *Gateway-RAG* under GPT-5, averaging results over five scenarios to observe how `Tokens_in` scales with environment size.

VI. Results

A. GPT-5 Benchmarks

Across the 20-scenario benchmark, gateway-based architectures achieve substantially higher task effectiveness than the CrewAI baselines on the evaluated models, namely GPT-5, GPT-5-mini, and GPT-5-nano. With GPT-5 as their backbone, both *Gateway* and *Gateway-RAG* achieve high success scores, indicating that the unified capability gateway yields reliable tasks completion. The simple pipelines on the other hand perform rather poorly: *Simple* is excluded from the reported results, as it did not produce meaningful results for comparison. Reasons are discussed in the following section. While *Simple-Crew* is able to complete end-to-end pipeline runs, it performs worst across all reported metrics. The difference in performance compared to the gateway-based architectures is most observable in score, with an absolute gap of nearly 90%. Moreover, while achieving roughly similar input token consumption, the token output of gateway-based architectures is greatly reduced, suggesting less framework-specific verbosity and more efficient agent handoffs. Finally, *Gateway* outperforms the retrieval-augmented variant across all metrics.

	Latency	Tokens In	Tokens Out	Task Score
Simple	N/A	N/A	N/A	N/A
Simple-Crew	270.01s	67.52k	52.24k	3.44%
Gateway	**188.45s**	**61.51k**	**14.21k**	**93.18%**
Gateway-RAG	223.04s	67.27k	15.50k	91.09%

Evaluation Results for GPT-5

B. Model Comparison

Across all tested models, the CrewAI baselines remain ineffective. Notably, *Simple-Crew*'s quality degrades with increased model capacity: Starting from 14% task success rate with GPT-5-nano down to 3% with GPT-5, as observable in tables I, II, and III. Also, token consumption increases substantially as models grow larger.

The number of output tokens in both pipelines follows a distinct pattern: GPT-5-nano consumes the most tokens by far, GPT-5-mini drastically reduces consumption, and with GPT-5 it slightly increases again.

In contrast, *Gateway* scales strongly in task success with model capacity. Its performance rises from roughly 49% with GPT-5-nano to 89% with GPT-5-mini and above 93% with GPT-5, showing that the architecture already achieves very strong results with the mid-sized model in the compact environment.

Gateway-RAG also benefits from increased model capacity, but more gradually and from a much lower starting point. On GPT-5-nano, the retrieval-augmented variant even underperforms *Simple-Crew* in task success despite being more efficient on the other reported metrics. With GPT-5-mini and GPT-5, its task success improves substantially, yet it remains below *Gateway* across all tested backbones. This suggests that retrieval-augmented capability discovery is not beneficial in a perfectly pure environment.

C. Environment Scaling Results

As already described in subsection V-G, starting from the perfectly configured environment, we gradually increase the number of available tools. Under this growing capability load, both gateway-based variants remain consistent on all reported metrics, with one exception: *Gateway*'s input token consumption increases roughly proportional to the number of available tools, whereas *Gateway-RAG* remains nearly constant.

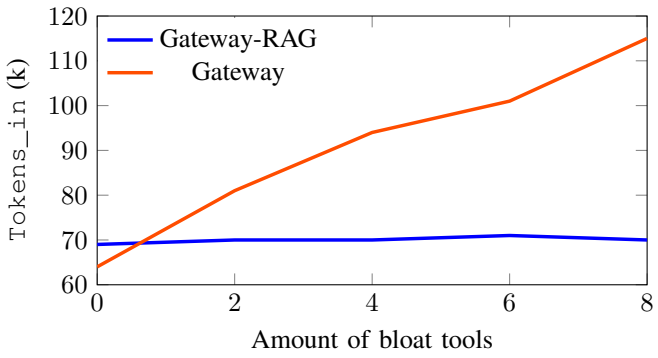


Fig. 5. `Tokens_in` under increasing capability load. Values are in thousands of tokens (k); the y-axis is truncated (60–120k) for readability.

VII. Discussion

A. Failure modes

The pipelines differ not only in final performance, but also in failure types. While the gateway-based architectures worst failures are vulnerability version range mismatches, the simple baselines fail earlier and in more fundamental ways.

This is also why the *Simple* baseline is excluded from the quantitative comparison: Most runs ended in timeout or pipeline abortion rather than successful task completion, which makes metrics such as latency or token consumption difficult to interpret in a meaningful way. The main issues in *Simple* were segmentation faults, likely related to the CrewAI stack and early library version. We also observed repeated tool-calling loops and parsing failures that led to stalling and dropped information.

Simple-Crew completed runs more reliably, but often got stuck in tool-calling loops, reaching the maximum number of turns. This likely contributes to its high latency and output token consumption. Also, tool agents being forced to give a final answer after exhausting the maximum number of invocations, even if relevant information could not be retrieved yet probably correlates with the pipelines low task success score.

B. Models, tokens, and verbosity

The model comparison shows that lower token consumption does not indicate better results. Across the gateway pipelines, output-token usage follows a consistent pattern: GPT-5-nano produces the largest outputs by far, GPT-5-mini reduces output drastically, and GPT-5 increases moderately again. This aligns with traces where the smallest model tends to over-generate and reiterate more, while the mid-sized model is more concise while retaining similar input token consumption.

Comparing output tokens and latency between pipelines also shows how long and erroneous the trajectory is. In the simple baselines, long trajectories can often be attributed to coordination issues. This is visible in *Simple-Crew*, whose output token consumption is unusually high compared to gateway-based pipelines, consistent with the observed extra coordination steps and verbose outputs being carried through the whole pipeline run. At the same time, *Simple-Crew* quality degrades as model capacity increases while token usage grows. Along tool calls occurring and max turns being reached, we have also observed that most individual runs simply miss notifications. Combining these two observations suggests that models capable of deep reasoning still can not fix suboptimal information injection strategy.

In contrast, gateway-based pipelines scale positively with model capacity and already score well with the mid-sized backbone. This supports the claim that mid-capacity models can be sufficient when planning-time information exposure is kept compact and execution details are pulled only when needed.

In addition, our traces also highlight another token cost driver: Delegation responses from downstream agents can greatly impact token usage in multi-agent chains (in some cases reaching tens of thousands of tokens), due to another unnecessary model generation when receiving the delegation response.

C. Retrieval

With the smallest model, *Gateway-RAG* underperforms the non-retrieval gateway variant. A plausible explanation is that retrieval-based discovery adds extra cognitive workload (query formulation, selection, verification) that small models might not be able to reliably manage. With GPT-5-mini and GPT-5, retrieval becomes much more effective. However, the results indicate that the non-retrieval gateway variant is better suited for small, well-configured environments.

D. Scaling

The capability-load experiment shows where retrieval becomes essential. As the ecosystem grows, *Gateway*'s input token consumption increases roughly proportional to the number of available tools, whereas *Gateway-RAG* remains nearly constant. This scaling trend is in line with prior work on retrieval-based agent systems [10, 8, 7, 18, 17, 9]. Hence, preventing irrelevant capabilities from entering the planning context mitigates context bloat. Although related literature finds that long contexts are directly correlated with poor performance, our findings do not match this claim, which might result from not scaling aggressively enough to make the effect visible.

E. Why separation helps

We cannot isolate the exact causal impact of each framework mechanism on the improved performance of the gateway architectures. However, the observed results (section VI) are consistent with the idea that planning quality degrades when execution boilerplate, protocol handling, and irrelevant capability descriptions are mixed within the context window.

In our setting, all pipelines use the same underlying models and the same task and environment configuration. However, the baselines are significantly more prone to tool-calling loops, parsing issues, and stalling, alongside low task scores. This suggests that agents needing to handle protocols themselves, and the way information is exposed to the planner, negatively affects end-to-end performance.

These observations also match the related literature: long contexts degrade reliability, irrelevant information distracts the model, and incorrect capability retrieval is a major source of agent failure [15, 28, 20, 7].

VIII. Future Work

A next step is to introduce finer-grained intermediate architectures between our simple baselines and the full gateway setup. Our current comparison involves a relatively large architectural step, which makes it harder to attribute the observed gains to any single design choice. Future work should therefore minimize framework-specific differences between the two architecture families and introduce incremental variants that isolate individual success drivers more precisely.

At the same time, the gateway requires deeper investigation of its security properties before it can be considered for industry deployment. As protocol-based agent systems grow in scope, authentication, attack resistance, and trust boundaries become critical concerns that should be addressed as part of future gateway research.

Our evaluation also revealed the lack of a standardized benchmark for multi-agent systems that operate across multiple protocols. Existing benchmarks focus primarily on MCP tool retrieval [20, 19], which does not capture the coordination and protocol-switching demands of gateway-based setups. We therefore had to design our own task environment to evaluate the different pipeline architectures. A dedicated multi-protocol benchmark would benefit future research in this area considerably.

On the system side, the gateway could be extended with more specialized retrieval methods. Prior work already explores promising tool-retrieval strategies [10, 7, 8, 18, 17]. Recent work also shows that specialized retrieval models can improve tool retrieval further [16].

Finally, our token consumption metric currently suffers from excessive upstream token usage caused by results being passed back through the whole agent chain without contributing additional value. Future work should investigate strategies to reduce this overhead, as it inflates measured costs in multi-agent pipelines like ours and limits the scalability of deeper delegation chains.

IX. Conclusion

Multi-agent systems increasingly rely on protocols such as MCP and A2A. However, current approaches tend to expose execution-level details directly into the agent's planning context. This forces agents to reason simultaneously about task semantics and protocol mechanics, increasing token consumption, latency, and failure rates.

In this work, we formalize this as a problem of poorly staged information exposure and address it with a unified capability gateway that abstracts over both tools and peer agents. The gateway keeps planning focused on semantic capability information and delays execution-specific details until they are actually needed. Our empirical evaluation showed that the gateway architecture consistently improves token efficiency, latency, and task success. The retrieval-based gateway variant becomes especially useful as the environment grows, preventing irrelevant information from entering the context window.

Taken together, these results suggest that future agent systems should be designed around staged information exposure rather than upfront capability dumping. The gateway architecture reinforces this principle while additionally providing a central point for configuration and governance across protocols. As multi-agent ecosystems continue to grow, this separation should become increasingly important for building reliable and scalable systems.

References

[1] Arash Ahmadi, Sarah Sharif, and Yaser M. Banad. *MCP Bridge: A Lightweight, LLM-Agnostic RESTful Proxy for Model Context Protocol Servers*. 2026. arXiv: 2504.08999 [cs.CR]. URL: https://arxiv.org/abs/2504.08999.

[2] Anthropic. *Introduction to Model Context Protocol (MCP)*. Accessed: 2026-03-03. 2024. URL: https://modelcontextprotocol.io/introduction.

[3] CrewAI Inc. *A2A Agent Delegation*. CrewAI Documentation. Accessed: 2026-03-06. 2026. URL: https://docs.crewai.com/en/learn/a2a-agent-delegation.

[4] CrewAI Inc. *CrewAI Documentation*. Online documentation. Accessed: 2026-03-06. 2026. URL: https://docs.crewai.com/.

[5] Peng Ding. *ToolRegistry: A Protocol-Agnostic Tool Management Library for Function-Calling LLMs*. 2025. arXiv: 2507.10593 [cs.SE]. URL: https://arxiv.org/abs/2507.10593.

[6] Abul Ehtesham et al. *A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)*. 2025. arXiv: 2505.02279 [cs.AI]. URL: https://arxiv.org/abs/2505.02279.

[7] Xiang Fei, Xiawu Zheng, and Hao Feng. *MCP-Zero: Active Tool Discovery for Autonomous LLM Agents*. 2025. arXiv: 2506.01056 [cs.AI]. URL: https://arxiv.org/abs/2506.01056.

[8] Tiantian Gan and Qiyao Sun. *RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation*. 2025. arXiv: 2505.03275 [cs.AI]. URL: https://arxiv.org/abs/2505.03275.

[9] Nishant Gaurav et al. *CodeMem: Architecting Reproducible Agents via Dynamic MCP and Procedural Memory*. 2025. arXiv: 2512.15813 [cs.SE]. URL: https://arxiv.org/abs/2512.15813.

[10] Nishant Gaurav et al. *Dynamic ReAct: Scalable Tool Selection for Large-Scale MCP Environments*. 2025. arXiv: 2509.20386 [cs.SE]. URL: https://arxiv.org/abs/2509.20386.

[11] Google. *Agent2Agent (A2A) Protocol Documentation*. Accessed: 2026-03-03. 2024. URL: https://google.github.io/A2A/.

[12] jakeazcona. *CrewAI Slices, Prompts and Control*. CrewAI Community (Discourse forum post). Accessed: 2026-03-05. June 2025. URL: https://community.crewai.com/t/crewai-slices-prompts-and-control/6166.

[13] R. Kishimoto. *A Dataset of Software Bill of Materials for Evaluating SBOM Consumption Tools*. Zenodo, Feb. 5, 2025. DOI: 10.5281/zenodo.14808410. URL: https://doi.org/10.5281/zenodo.14808410 (visited on 03/03/2026).

[14] Naveen Krishnan. *Advancing Multi-Agent Systems Through Model Context Protocol: Architecture, Implementation, and Applications*. 2025. arXiv: 2504.21030 [cs.MA]. URL: https://arxiv.org/abs/2504.21030.

[15] Nelson F. Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. 2023. arXiv: 2307.03172 [cs.CL]. URL: https://arxiv.org/abs/2307.03172.

[16] Xuan Lu et al. *Tools are under-documented: Simple Document Expansion Boosts Tool Retrieval*. 2025. arXiv: 2510.22670 [cs.CL]. URL: https://arxiv.org/abs/2510.22670.

[17] Elias Lumer et al. *ScaleMCP: Dynamic and Auto-Synchronizing Model Context Protocol Tools for LLM Agents*. 2025. arXiv: 2505.06416 [cs.CL]. URL: https://arxiv.org/abs/2505.06416.

[18] Elias Lumer et al. *Tool-to-Agent Retrieval: Bridging Tools and Agents for Scalable LLM Multi-Agent Systems*. 2025. arXiv: 2511.01854 [cs.CL]. URL: https://arxiv.org/abs/2511.01854.

[19] Ziyang Luo et al. *MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers*. 2025. arXiv: 2508.14704 [cs.AI]. URL: https://arxiv.org/abs/2508.14704.

[20] Guozhao Mo et al. *LiveMCPBench: Can Agents Navigate an Ocean of MCP Tools?* 2026. arXiv: 2508.01780 [cs.AI]. URL: https://arxiv.org/abs/2508.01780.

[21] Xiaoyi Nie and Haitao Zhang. "Large Language Models Tool Retrieval and Context Compression via Dynamic Graph-Based Relation Modeling". In: *Academic Journal of Computing & Information Science* (2025). URL: https://api.semanticscholar.org/CorpusID:281120892.

[22] Faheem Nizar et al. *Agent-as-a-Graph: Knowledge Graph-Based Tool and Agent Retrieval for LLM Multi-Agent Systems*. 2025. arXiv: 2511.18194 [cs.CL]. URL: https://arxiv.org/abs/2511.18194.

[23] OSV.dev Contributors. *OSV API (v1.0) Documentation*. Online documentation. Accessed: 2026-03-06. 2026. URL: https://google.github.io/osv.dev/api/.

[24] Shishir G. Patil et al. *Gorilla: Large Language Model Connected with Massive APIs*. 2023. arXiv: 2305.15334 [cs.CL]. URL: https://arxiv.org/abs/2305.15334.

[25] Python Packaging Authority (PyPI). *crewai: PyPI package page and release history*. Online. Accessed: 2026-03-06. 2026. URL: https://pypi.org/project/crewai/.

[26] Yujia Qin et al. *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs*. 2023. arXiv: 2307.16789 [cs.AI].

[27] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. "AI Agents vs. Agentic AI: A Conceptual taxonomy, applications and challenges". In: *Information Fusion* 126 (Feb. 2026), p. 103599. ISSN: 1566-2535. DOI: 10.1016/j.inffus.2025.103599. URL: http://dx.doi.org/10.1016/j.inffus.2025.103599.

[28] Freda Shi et al. *Large Language Models Can Be Easily Distracted by Irrelevant Context*. 2023. arXiv: 2302.00093 [cs.CL]. URL: https://arxiv.org/abs/2302.00093.

[29] Theodore R. Sumers et al. *Cognitive Architectures for Language Agents*. 2024. arXiv: 2309.02427 [cs.AI]. URL: https://arxiv.org/abs/2309.02427.

[30] Renxi Wang et al. *ToolGen: Unified Tool Retrieval and Calling via Generation*. 2025. arXiv: 2410.03439 [cs.CL]. URL: https://arxiv.org/abs/2410.03439.

[31] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: 2210.03629 [cs.CL]. URL: https://arxiv.org/abs/2210.03629.

APPENDIX A
MODEL COMPARISON TABLES

TABLE I
EVALUATION RESULTS FOR GPT-5-NANO

	Latency	Tokens In	Tokens Out	Task Score
Simple	N/A	N/A	N/A	N/A
Simple-Crew	219.07s	12.93k	10.26k	14.69%
Gateway	305.18s	66.34k	26.52k	**49.06%**
Gateway-RAG	**192.18s**	**52.86k**	**23.69k**	10.16%

TABLE II
EVALUATION RESULTS FOR GPT-5-MINI

	Latency	Tokens In	Tokens Out	Task Score
Simple	N/A	N/A	N/A	N/A
Simple-Crew	328.19s	68.11k	40.37k	12.08%
Gateway	**99.34s**	60.72k	**9.86k**	**89.27%**
Gateway-RAG	119.00s	**59.30k**	10.49k	61.82%

TABLE III
EVALUATION RESULTS FOR GPT-5

	Latency	Tokens In	Tokens Out	Task Score
Simple	N/A	N/A	N/A	N/A
Simple-Crew	270.01s	67.52k	52.24k	3.44%
Gateway	**188.45s**	**61.51k**	**14.21k**	**93.18%**
Gateway-RAG	223.04s	67.27k	15.50k	91.09%